КРИПТОГРАФІЯ

КОМП’ЮТЕРНИЙ ПРАКТИКУМ №4

Вивчення криптосистеми RSA та алгоритму електронного підпису; ознайомлення з методами генерації параметрів для асиметричних криптосистем

ФБ-43 Цвілій Влада

Варіант 23

Мета та основні завдання роботи

Ознайомлення з тестами перевірки чисел на простоту і методами генерації ключів для асиметричної криптосистеми типу RSA; практичне ознайомлення з системою захисту інформації на основі криптосхеми RSA, організація з використанням цієї системи засекреченого зв'язку й електронного підпису, вивчення протоколу розсилання ключів.

Порядок виконання роботи

1. Написати функцію пошуку випадкового простого числа з заданого інтервалу або заданої довжини, використовуючи датчик випадкових чисел та тести перевірки на простоту.

2. За допомогою цієї функції згенерувати дві пари простих чисел p, q і p_1, q_1 довжини щонайменше 256 біт.

3. Написати функцію генерації ключових пар для RSA. Для однієї зі згенерованих пар ключів навести значення основних параметрів та перевірити виконання співвідношень, необхідних для коректної роботи RSA.

4. Написати програму шифрування, розшифрування і створення повідомлення з цифровим підписом для абонентів A і B. За допомогою датчика випадкових чисел вибрати відкрите повідомлення M і знайти криптограму для абонентів A і B, перевірити правильність розшифрування. Скласти для A і B повідомлення з цифровим підписом і перевірити його.

5. За допомогою раніше написаних на попередніх етапах програм організувати роботу протоколу конфіденційного розсилання ключів з підтвердженням справжності по відкритому каналу за допомогою алгоритму RSA.

Хід роботи

```python
import secrets

#  1. Математика

def power_mod(b, e, m):
    """b^e mod m (схема Горнера)."""
    r = 1; b %= m
    while e:
        if e & 1: r = r * b % m
        b = b * b % m; e >>= 1
    return r

def ext_gcd(a, b):
    if not b: return a, 1, 0
    g, x, y = ext_gcd(b, a % b)
    return g, y, x - (a // b) * y

def mod_inv(e, phi):
    g, x, _ = ext_gcd(e, phi)
    if g != 1: raise ValueError("Оберненого не існує")
    return x % phi
```

1. Функція power_mod(b, e, m)
Виконує швидке обчислення виразу b^e mod m за схемою Горнера.

2. Функція ext_gcd(a, b)
Рекурсивно реалізує розширений алгоритм Евкліда.

3. Функція mod_inv(e, phi)
Знаходить секретну експоненту d, обернену до відкритої експоненти e за модулем phi, розв'язує рівняння (e * d) mod phi == 1.

```python
# 4.py > trial_division
#  2. Тести простоти

SMALL_PRIMES = [2,3,5,7,11,13,17,19,23,29,31,37,41,43,47,53,59,61,67,71,
                73,79,83,89,97,101,103,107,109,113,127,131,137,139,149,151,
                157,163,167,173,179,181,191,193,197,199,211,223,227,229,233,
                239,241,251,257,263,269,271,277,281,283,293,307,311,313,317]

def trial_division(n):
    for p in SMALL_PRIMES:
        if n == p: return True
        if n % p == 0: return False
    return True

def miller_rabin(p, k=40):
    if p < 2: return False
    if p in (2, 3): return True
    if p % 2 == 0: return False
    d, s = p - 1, 0
    while d % 2 == 0: d //= 2; s += 1
    for _ in range(k):
        x = secrets.randbelow(p - 3) + 2
        xr = power_mod(x, d, p)
        if xr in (1, p - 1): continue
        for _ in range(1, s):
            xr = xr * xr % p
            if xr == p - 1: break
            if xr == 1: return False
        else: return False
    return True
```

trial_division(n) - тест пробних ділень:
Бере список перших простих чисел до 317 і ділить нашого кандидата на кожне з них. Якщо число поділилося хоч на одне -воно одразу відкидається як складене.

miller_rabin(p, k=40) - тест Міллера-Рабіна на 40 кроків з різними основами

```python
def miller_rabin_demo(p, base=2):
    """Проміжні результати одного раунду для звіту."""
    print(f"\n--- Тест Міллера-Рабіна за основою {base} ---")
    d, s = p - 1, 0
    while d % 2 == 0: d //= 2; s += 1
    print(f"p-1 = d·2^s  =>  d={d}, s={s}")
    xr = power_mod(base, d, p)
    print(f"Крок 2.1: x_0 = {base}^d mod p = {xr}")
    if xr in (1, p - 1):
        print(f"=> сильно псевдопросте (x_0 == {'1' if xr==1 else '-1'})")
        return
    for r in range(1, s):
        xr = xr * xr % p
        print(f"Крок 2.2 r={r}: x_{r} = {xr}")
        if xr == p - 1: print(f"=> сильно псевдопросте (x_{r} == -1)"); return
        if xr == 1:     print(f"=> СКЛАДЕНЕ (x_{r} == 1)"); return
    print("=> СКЛАДЕНЕ (жодного -1)")
```

miller_rabin_demo(p, base=2) виконує демонстраційний раунд тестування для обраного простого числа за фіксованою основою 2 та покроково виводить у консоль проміжні значення

```
71  #  3. Генерація простих чисел
72
73  def get_prime(bits=256, rejected=None):
74      while True:
75          n = secrets.randbits(bits) | (1 << (bits - 1)) | 1
76          if not trial_division(n):
77              if rejected is not None and len(rejected) < 5:
78                  rejected.append((n, "пробні ділення"))
79              continue
80          if not miller_rabin(n):
81              if rejected is not None and len(rejected) < 5:
82                  rejected.append((n, "Міллер-Рабін"))
83              continue
84          return n
85
```

Створення випадкового числа і перевірка його за допомогою функцій тесту пробного ділення та тесту Міллера-Рабіна. Якщо кандидат не проходить будь-який з етапів, інформація про нього (значення та причина відхилення) записується у список rejected для відображення у звіті, після чого генерується новий кандидат. Процес триває доти, доки не буде знайдено число, що успішно пройшло обидва етапи перевірки.

```
85
86  #  4. Обов'язкові процедури
87
88  def GenerateKeyPair(bits=256, rejected=None):
89      e = 65537
90      while True:
91          p = get_prime(bits, rejected)
92          q = get_prime(bits, rejected)
93          phi = (p - 1) * (q - 1)
94          if p != q and ext_gcd(e, phi)[0] == 1:
95              break
96      n = p * q
97      return (e, n), (mod_inv(e, phi), p, q)
98
```

```python

def Encrypt(M, e, n):   return power_mod(M, e, n)
def Decrypt(C, d, n):   return power_mod(C, d, n)
def Sign(M, d, n):      return power_mod(M, d, n)
def Verify(M, S, e, n): return M == power_mod(S, e, n)

def SendKey(k, d_A, n_A, e_B, n_B):
    if n_A > n_B or not (0 < k < n_A):
        raise ValueError("Порушено умови протоколу")
    k_1 = Encrypt(k, e_B, n_B)
    S_1 = Encrypt(Sign(k, d_A, n_A), e_B, n_B)
    return k_1, S_1

def ReceiveKey(k_1, S_1, d_B, n_B, e_A, n_A):
    k = Decrypt(k_1, d_B, n_B)
    return k, Verify(k, Decrypt(S_1, d_B, n_B), e_A, n_A)

```

Генерація ключів і реалізація процедур: GenerateKeyPair(), Encrypt(), Decrypt(), Sign(), Verify(), SendKey(), ReceiveKey()

```python

if __name__ == "__main__":
    SEP = "=" * 70

    # [1] Генерація простого числа + відхилені кандидати
    bad = []
    p_demo = get_prime(256, bad)
    print(SEP)
    print(f"[1] Просте p ({p_demo.bit_length()} біт):\n{hex(p_demo)}")
    print("Відхилені кандидати:")
    if not bad: print("  (немає — число знайдено з першої спроби)")
    for i, (c, r) in enumerate(bad, 1): print(f"  {i}. {hex(c)[:20]}... ({r})")
    miller_rabin_demo(p_demo, base=2)

```

```python
# [2] Генерація ключів A і B з умовою n_A <= n_B
print(f"\n{SEP}\n[2] Генерація ключів RSA для A і B...")
bad_A, bad_B = [], []
pub_A, priv_A = GenerateKeyPair(256, bad_A)
pub_B, priv_B = GenerateKeyPair(256, bad_B)

if pub_A[1] > pub_B[1]:
    pub_A, priv_A, bad_A, pub_B, priv_B, bad_B = (
        pub_B,
        priv_B,
        bad_B,
        pub_A,
        priv_A,
        bad_A,
    )

e_A, n_A = pub_A
d_A, p_A, q_A = priv_A
e_B, n_B = pub_B
d_B, p_B, q_B = priv_B

phi_A = (p_A - 1) * (q_A - 1)
phi_B = (p_B - 1) * (q_B - 1)

print("Абонент A:")
print(f"  p  = {hex(p_A)}")
print(f"  q  = {hex(q_A)}")
print(f"  n  = {hex(n_A)}")
print(f"  e  = {e_A}")
print(f"  d  = {hex(d_A)}")
print("  Відхилені кандидати при генерації для A:")
for i, (c, r) in enumerate(bad_A, 1):
    print(f"    {i}. {hex(c)[:20]}... ({r})")
```

```python
print("\nАбонент B:")
print(f"  p1 = {hex(p_B)}")
print(f"  q1 = {hex(q_B)}")
print(f"  n1 = {hex(n_B)}")
print(f"  e1 = {e_B}")
print(f"  d1 = {hex(d_B)}")
print("  Відхилені кандидати при генерації для B:")
for i, (c, r) in enumerate(bad_B, 1):
    print(f"    {i}. {hex(c)[:20]}... ({r})")

print("\nПеревірки:")
print(f"  n_A <= n_B: {n_A <= n_B}")
print(f"  (e_A * d_A) mod phi_A == 1: {(e_A * d_A) % phi_A == 1}")
print(f"  (e_B * d_B) mod phi_B == 1: {(e_B * d_B) % phi_B == 1}")
```

[1]

Функція get_prime генерує 256-бітне число p, паралельно записуючи у список bad перших кандидатів, які завалили перевірку.

Виводить довжину числа, його значення в hex та список невдалих спроб із причиною відхилення.

Викликає miller_rabin_demo, який покроково друкує проміжні результати розкладу

[2] генерація ключів для А і Б

```python
# [3] Шифрування / розшифрування
print(f"\n{SEP}\n[3] Шифрування і розшифрування...")
M = secrets.randbelow(n_A - 1) + 1

# A шифрує для B
C_forB = Encrypt(M, e_B, n_B)
print(f"A -> B: M={hex(M)}\n  C={hex(C_forB)}\n  Розшифровано вірно: {Decrypt(C_forB, d_B, n_B) == M}")

# B шифрує для A
C_forA = Encrypt(M, e_A, n_A)
print(f"B -> A: M={hex(M)}\n  C={hex(C_forA)}\n  Розшифровано вірно: {Decrypt(C_forA, d_A, n_A) == M}")

# [4] Цифровий підпис
print(f"\n{SEP}\n[4] Цифровий підпис...")
S_A = Sign(M, d_A, n_A)
print(f"Підпис A: S={hex(S_A)}\n  Вірний: {Verify(M, S_A, e_A, n_A)}")
S_B = Sign(M, d_B, n_B)
print(f"Підпис B: S={hex(S_B)}\n  Вірний: {Verify(M, S_B, e_B, n_B)}")

# [5] розсилання ключів A -> B
print(f"\n{SEP}\n[5] Протокол розсилання ключів A -> B...")
k = secrets.randbelow(n_A - 2) + 1
k_1, S_1 = SendKey(k, d_A, n_A, e_B, n_B)
k_recv, valid = ReceiveKey(k_1, S_1, d_B, n_B, e_A, n_A)
print(f"k    = {hex(k)}")
print(f"k_1  = {hex(k_1)}")
print(f"S_1  = {hex(S_1)}")
print(f"k отримано: {hex(k_recv)}")
print(f"Ключі збігаються: {k == k_recv} | Підпис A підтверджено: {valid}")
print(SEP)
```

[3] Генерація випадкового повідомлення M і шифрування/розшифрування його.

[4] Формування підписів користувачів і їхня перевірка.

[5] Безпечна передача секретного ключа.

Результат роботи програми:

```
  ================================================================
  [1] Просте p (256 біт):
  0xaca8a704d4d90637029b6329727a7100fec2bacd8d131875cd7822f9debde295
  Відхилені кандидати:
    1. 0xb0d1c44db13e7e0eeb... (пробні ділення)
    2. 0x9dab238e8eeb1778a1... (Міллер-Рабін)
    3. 0xc7a9dd0a29eacfa92b... (пробні ділення)
    4. 0xa8208f634e8f232e8d... (пробні ділення)
    5. 0xbba75d3ae851e31908... (Міллер-Рабін)

  --- Тест Міллера-Рабіна за основою 2 ---
  p-1 = d·2^s  =>  d=1952394824627494312580959833310528727257629660995217451442490524779943136272 5, s=2
  Крок 2.1: x_0 = 2^d mod p = 42666983136334774102112998801130554272093429471649090356339780431796815804747
  Крок 2.2 r=1: x_1 = 78095792985099772503238393332421149090305186439808698057699620991197725450900
  => сильно псевдопросте (x_1 == -1)

  ================================================================
```

Успішно знайдено 256-бітне просте число p. Також зафіксовано перших 5 згенерованих випадкових кандидатів, які не пройшли перевірку: більшість була миттєво відсіяна на етапі пробних ділень, а інші - на етапі тесту Міллера-Рабіна. Це демонструє ефективність дворівневої фільтрації.

Також для знайденого числа p наведено протокол одного раунду перевірки за основою 2. Оскільки умова появи -1 виконалася на першій же ітерації піднесення до квадрата, алгоритм зафіксував виконання критерію: число визнано сильно псевдопростим за цією основою.

```
  ================================================================
  [2] Генерація ключів RSA для A і B...
  Абонент A:
    p  = 0x8705b5d2af0441cc0d38362258dc3116ab72ed0887a1127279fa2d7b74ad2eb1
    q  = 0xc7b4d159d2c5b780214a3131ac892519a4542d8b5aadbcec62fe47580fa59963
    n  = 0x6954cec5a7c78c999c080312d459fdee43e996de71d2390b84e56ecaac6c2832e514f3abc02fd5aabd734bc5c5871259801d4b1e8991cac1766fdcad16f5d773
    e  = 65537
    d  = 0x4d0afb3317c59dd1bd8f1eb067226bda1b528047b4794fc05d3474154a88d1dd735c307fe1e41f8c0a81a8968b3221aefeec0c155ec25f9d7f231792cceef81
    Відхилені кандидати при генерації для A:
      1. 0xfa9375d5e7d4b879bf... (пробні ділення)
      2. 0xcaff94b8e7207c3c7f... (Міллер-Рабін)
      3. 0xcc0e60285f124c6e80... (пробні ділення)
      4. 0xac3d00aa23247e49eb... (пробні ділення)
      5. 0xf5df9f2cbc2bdd6d9a... (Міллер-Рабін)

  Абонент B:
    p1 = 0xa139ad93a0ef47d5812ef1f1c2ae7495f4d9234e467b1167aa15a5eff8088b3f
    q1 = 0xd56128e92856bdb1cf7778a7cd4200e6e3ee3727275b1cd414b628270f34c38d
    n1 = 0x86622e0a73dcc1e8aad1153eee83679efb069481eae53d99444aec34be6e132d2b28662f34a151103d2f7de1a3edd85a27ecf03d9df85935f9721f981891aeb3
    e1 = 65537
    d1 = 0x3c74f719a4be704e686ae0100be5fa3c3fdad9fd08dc51361ecf462d046f2918c485db5bfd88791704b336857ad8c81da079dc44cfc2cf6afbefcf432db4b3e1
    Відхилені кандидати при генерації для B:
      1. 0x9c07a3b715dbf49a7e... (пробні ділення)
      2. 0xff2ebbf8b93493de9c... (пробні ділення)
      3. 0xe97ae8d0f15ff9ed9b... (пробні ділення)
      4. 0x935285416cdcf7d20d... (пробні ділення)
      5. 0xf674d0f0360344aa9f... (пробні ділення)

  Перевірки:
    n_A <= n_B: True
    (e_A * d_A) mod phi_A == 1: True
    (e_B * d_B) mod phi_B == 1: True
```

Генерація ключів для A і B.

Таблиця згенерованих простих чисел

Абонент	Параметр	Скорочене значення	Розмірність
Абонент A	p	`0x8705b5d2af04...`	256 біт
Абонент A	q	`0xc7b4d159d2c5...`	256 біт
Абонент B	p_1	`0xa139ad93a0ef...`	256 біт
Абонент B	q_1	`0xd56128e92856...`	256 біт

Для кожного з абонентів було згенеровано по два незалежних 256-бітних простих числа (p, q для абонента A та p1, q1 для абонента B). У процесі пошуку зафіксовано кандидатів, які не пройшли перевірку: більшість відсіялася на етапі пробних ділень, а решта - тестом Міллера-Рабіна, що підтверджує коректну роботу фільтрів для обох абонентів.

Демонстрація роботи процедур Encrypt та Decrypt:

```
============================================================
[3] Шифрування і розшифрування...
A -> B: M=0x326f405ca8ef70795887470b086c8f044ad8b261c3d9f63c0254cc488c7c0738bf69f35f9f4442531e092aa2d98c477f857ad8d6a0b4b95679a58230fc4473c7
  C=0x6ed17e7a537b0a95b022d472fb7b29239461e3c7ecf340f29f0a0068c12655a0fdd31863c9834607c5d24bc805ffe992c94eee63e7a8b2c06e94ff85ef620fca
  Розшифровано вірно: True
B -> A: M=0x326f405ca8ef70795887470b086c8f044ad8b261c3d9f63c0254cc488c7c0738bf69f35f9f4442531e092aa2d98c477f857ad8d6a0b4b95679a58230fc4473c7
  C=0x539c9687e69136c1f42bb48b042db4932879c5df0ea378530bd9b94e3039e0deba46645bae26474d3a68ba9c93377bcbe8cc91cc7da33622ee62dc46094c620c
  Розшифровано вірно: True
```

За допомогою криптографічного датчика випадкових чисел обрано відкрите повідомлення M, розмір якого строго менший за модулі обох абонентів. Абонент A зашифрував повідомлення на відкритому ключі абонента B, у результаті чого отримано шифротекст C. Абонент B виконав розшифрування на власному секретному ключі. Порівняння розшифрованого тексту з початковим повернуло True.

Аналогічно абонент B зашифрував те саме повідомлення відкритим ключем абонента, а A успішно розшифрував його ключем d_A (отримано True).

Попри те, що для обох операцій використовувалося одне й те саме вихідне повідомлення M, отримані шифротексти C кардинально відрізняються, оскільки вони обчислювалися за різними відкритими ключами та різними модулями.

Процедури формування та верифікації цифрового підпису RSA для обох абонентів:

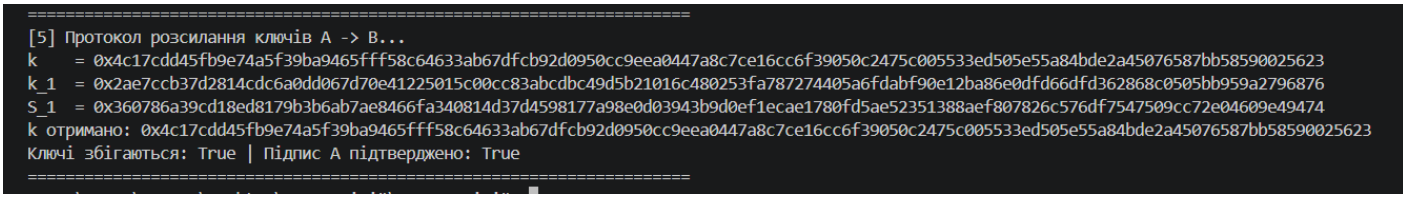
```
============================================================
[4] Цифровий підпис...
Підпис A: S=0x469fc2169a137461d931e9ee5171878276471f7fc4a7977451b213b471c165c6646efe006dbc4efaea678eed073e3243b7e6c388591f54ac874f8a224130cecb
  Вірний: True
Підпис B: S=0x54b995ae5174dbd85b298544c10ec9932d4228048e6267ea6efc21bdfd4f6e3795f089a7b0f6071bb6affca8cf8d6c357e190b8d7dfd11e35fa3f69ea88ef681
  Вірний: True

============================================================
```

Для повідомлення M кожен абонент створив власний цифровий підпис за формулою S = M^d mod n:

- Абонент A підписав повідомлення особистим секретним ключем d_A.
- Абонент B підписав повідомлення особистим секретним ключем d_B.

Перевірка здійснювалася за допомогою відкритого ключа відповідного відправника (e, n) і в обох випадках процедура Verify повернула результат True.



Продемонстровано роботу протоколу розподілу ключів, за яким абонент A передає абоненту B ключ k у вигляді криптограми k_1 та зашифрованого підпису S_1. Абонент B успішно відновив вихідний ключ k та підтвердив справжність відправника. Конфіденційність передачі забезпечено використанням ключів отримувача, а автентифікацію автора - підписом на секретному ключі відправника d_A та його верифікацією відкритим ключем e_A.

Труднощі, що виникли в ході роботи та їхнє розв’язання

1. Обчислення великих чисел та переповнення пам'яті:

При роботі з 256-бітними та 512-бітними числами використання звичайного оператора піднесення до степеня ** призводило до спроби створити числа з мільйонами знаків, що вичерпувало оперативну пам'ять і зависало програму. Тому було власноруч реалізовано функцію швидкого піднесення до степеня за модулем power_mod (схему Горнера). Завдяки тому, що взяття залишку % m виконується на кожному кроці піднесення до квадрата та множення, проміжні значення ніколи не перевищують величину m^2, що забезпечило високу швидкість і мінімальне використання пам'яті.

2. Неможливість повної перевірки простоти методом ділення:

Для 256-бітного числа неможливо перевірити простоту звичайним перебором дільників, оскільки перевірка чисел аж до кореня з кандидата (до 2^128) вимагала б величезного часу обчислень. Тому було застосовано дворівневий комбінований підхід. Перший етап (trial_division) перевіряє подільність лише на перші малі прості числа, що майже миттєво відсіює до 90% складених чисел. Для тих кандидатів, що пройшли першу перевірку, запускається повноцінний імовірнісний тест простоти.

3. Вибір та реалізація тесту Міллера-Рабіна замість тесту Ферма:

Простіший тест Ферма має суттєвий недолік - існування чисел Кармайкла (складених чисел, які прикидаються простими і дають залишок 1 на фініші). Тому було реалізовано тест Міллера-Рабіна, який покроково підносить основу до квадрата і перевіряє наявність значення -1 (тобто p - 1) безпосередньо перед отриманням 1. Використання k = 40 незалежних випадкових основ дозволило звести ймовірність помилки до нехтовно малої величини.

4. Представлення великих чисел у консолі:

У десятковій системі числа довжиною 512 біт містять понад 150 знаків. Їхнє виведення захаращувало термінал, робило лог нечитабельним і вкрай незручним для візуального аналізу. Тому усі вихідні та проміжні числові дані було переведено у шістнадцятковий формат. Це скоротило довжину запису майже втричі та зробило представлення ключів читабельним.

5. Забезпечення обов'язкової умови протоколу розсилання ключів (n_A <= n_B):

За протоколом RSA цифровий підпис абонента A шифрується на модулі абонента B. Якщо модулі згенеруються випадково так, що n_A > n_B, значення підпису може перевищити n_B. У такому разі взяття залишку безповоротно обріже дані, і B не зможе верифікувати підпис. Тому замість тривалого перегенерування ключів у циклі в програмі було реалізовано перевірку: якщо модуль A виявився більшим за модуль B, пари ключів абонентів автоматично міняються місцями, що миттєво забезпечує виконання умови n_A <= n_B.

6. Форматування даних при роботі з зовнішнім сервером:

Проблема: при взаємодії з веб-сервером виникали збої через невідповідність форматів: сервер очікував HEX-рядки виключно без префікса 0x, у верхньому регістрі та був чутливим до випадкових пробілів і переносів рядків при копіюванні. Тому було написано автономний допоміжний скрипт test_server.py, у якому реалізовано очищення введених рядків від пробілів методом .strip() та автоматичне приведення символів до верхнього регістру функцією to_hex(). Це дозволило успішно виконати як пряме шифрування для сервера, так і зворотне розшифрування.

Перевірка на сервері:

test_server.py > ...

```python
def power_mod(b, e, m):
    r = 1
    b %= m
    while e:
        if e & 1:
            r = r * b % m
        b = b * b % m
        e >>= 1
    return r

def to_hex(n):
    return hex(n)[2:].upper()
def from_hex(s):
    return int(s.strip(), 16)

def main():
    print("=" * 60)
    print("ПУЛЬТ ДЛЯ РОБОТИ З ТЕСТОВИМ СЕРВЕРОМ RSA")
    print("=" * 60)
    print("Оберіть операцію:")
    print("1. Зашифрувати повідомлення ДЛЯ сервера (Пункт А)")
    print("2. Розшифрувати шифротекст ВІД сервера (Пункт Б)")
    print("3. Вийти")
    print("=" * 60)

    choice = input("Ваш вибір (1/2/3): ").strip()

    if choice == "1":
        print("\n--- РЕЖИМ 1: Шифрування для сервера ---")
        mod_str = input("Вставте Modulus сервера: ")
        exp_str = input("Вставте Public exponent: ")
        msg_str = input(
            "Введіть ваше повідомлення в HEX (наприклад, 123456789ABCDEF): "
        )
```

```python
        n = from_hex(mod_str)
        e = from_hex(exp_str)
        M = from_hex(msg_str)

        C = power_mod(M, e, n)

        print("\n" + "-" * 50)
        print("ГОТОВО! Скопіюйте цей Ciphertext і вставте на сайті:")
        print(to_hex(C))
        print("-" * 50)

    elif choice == "2":
        print("\n--- РЕЖИМ 2: Розшифрування повідомлення від сервера ---")
        c_str = input("Вставте Ciphertext: ")
        d_str = input("Вставте ваш секретний ключ d: ")
        n_str = input("Вставте ваш модуль n: ")

        C = from_hex(c_str)
        d = from_hex(d_str)
        n = from_hex(n_str)

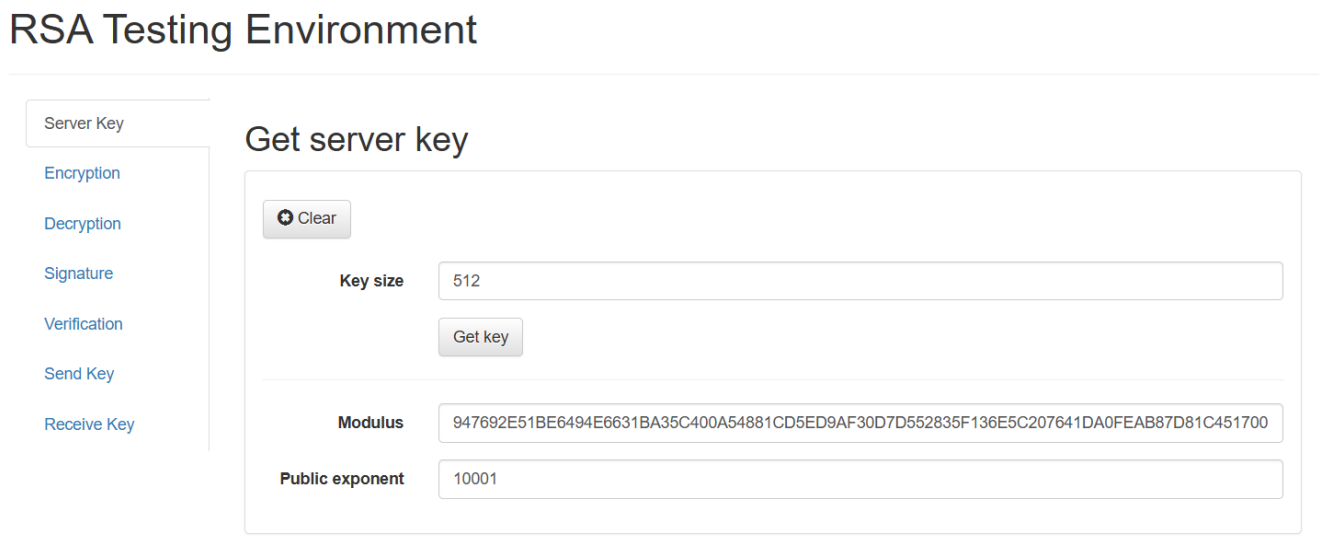
        M = power_mod(C, d, n)

        print("\n" + "-" * 50)
        print("РОЗШИФРОВАНО! Відкритий текст (HEX):")
        print(to_hex(M))
        print("-" * 50)
```

Програма є допоміжним інтерактивним інструментом для перевірки сумісності власної реалізації RSA з навчальним веб-сервером. Вона дозволяє в ручному режимі зашифрувати повідомлення на відкритому ключі сервера або розшифрувати отриманий від сервера шифротекст власним секретним ключем d. Усі дані автоматично очищаються від пробілів та конвертуються між числовим значенням і форматом HEX функціями to_hex та from_hex.

Для перевірки коректності власної реалізації алгоритму шифрування було виконано взаємодію з тестовим веб-сервером:

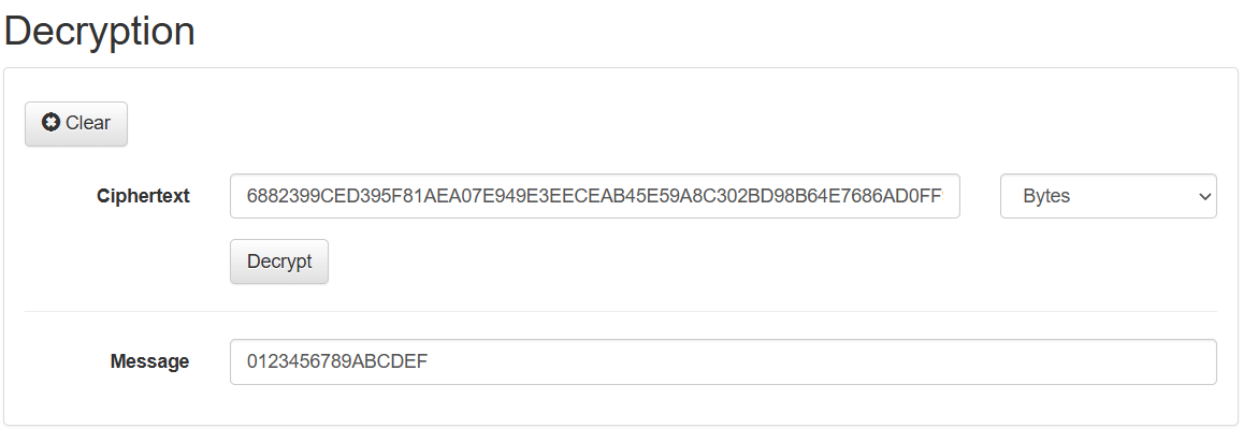
На сторінці «Server Key» було згенеровано пару ключів сервера довжиною 512 біт. Отримано відкритий модуль сервера та відкриту експоненту.




```
============================================================
ПУЛЬТ ДЛЯ РОБОТИ З ТЕСТОВИМ СЕРВЕРОМ RSA
============================================================
Оберіть операцію:
1. Зашифрувати повідомлення ДЛЯ сервера (Пункт А)
2. Розшифрувати шифротекст ВІД сервера (Пункт Б)
3. Вийти
============================================================
Ваш вибір (1/2/3): 1

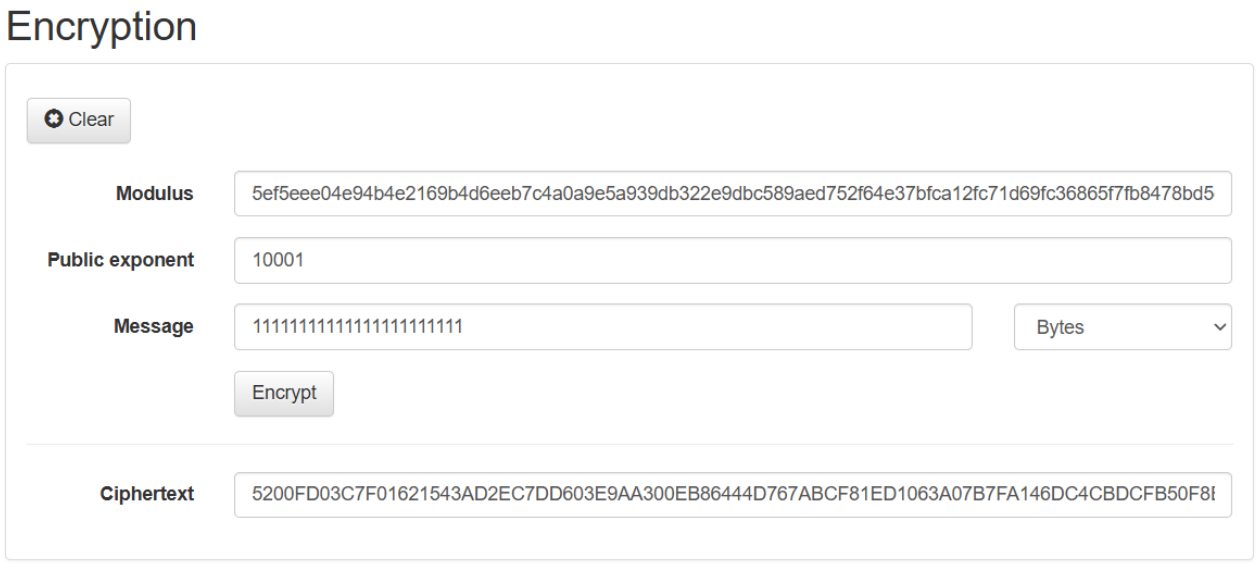
--- РЕЖИМ 1: Шифрування для сервера ---
Вставте Modulus сервера: 947692E51BE6494E6631BA35C400A54881CD5ED9AF30D7D552835F136E5C207641DA0FEAB87D81C451700FB846C3623A2863108AC0AFE58CA7266394EC0A436D
Вставте Public exponent: 10001
Введіть ваше повідомлення в HEX (наприклад, 123456789ABCDEF): 123456789ABCDEF

------------------------------------------------
ГОТОВО! Скопіюйте цей Ciphertext і вставте на сайті:
6882399CED395F81AEA07E949E3EECEAB45E59A8C302BD98B64E7686AD0FF9175EE2694C2A0E88A6AA6E230950759FBE31F3CBA29ECA04C20159C26636D6E8B
------------------------------------------------
PS C:\Users\Lenovo\Desktop\ФБ43_Цвілій\ФБ_43_Цвілій>
```



За допомогою допоміжної програми test_server.py виконано шифрування відкритого повідомлення M = 123456789ABCDEF на відкритому ключі сервера. Програма успішно обчислила значення шифротексту, який далі передається на сайт для перевірки розшифрування.

Перевірка операції розшифрування повідомлення від сервера:



(дані, що згенеровані для абонента A)

Сервер зашифрував тестове відкрите повідомлення M = 1111111111111111111111 та згенерував шифротекст.

Отриманий шифротекст було передано в створену програму для перевірки:



За допомогою секретного ключа d та модуля n абонента А програма успішно розшифрувала криптограму та відновила вихідне повідомлення 11111111111111111111.

Отже, обидва тести успішно підтвердили повну сумісність власної реалізації RSA з еталонним сервером для операцій шифрування та розшифрування.

Контрольні питання

1. Які перевірки виконуються програмою під час генерації простих чисел і для чого потрібна кожна з них?
Виконуються дві послідовні перевірки. Спочатку число примусово робиться непарним і проходить тест пробних ділень на малі прості числа, що дозволяє миттєво відсіяти до 90% складених кандидатів без важких обчислень. Далі запускається тест Міллера-Рабіна на 40 раундів із випадковими основами, який відсіює решту складених чисел та гарантує простоту з мізерним шансом на помилку.

2. Які умови повинні виконуватися для параметрів згенерованої пари ключів RSA? Як їх було перевірено у вашій реалізації?
Прості числа p і q мають бути різними (p != q), відкрита експонента e повинна бути взаємно простою зі значенням функції Ойлера phi = (p - 1)*(q - 1), тобто gcd(e, phi) == 1, а секретний ключ d має задовольняти умову (e * d) mod phi == 1. Для протоколу передачі ключів необхідне виконання умови n_A <= n_B. У програмі це перевірено через розширений алгоритм Евкліда під час генерації, автоматичну перестановку пар ключів для впорядкування модулів та пряму перевірку виразу (e * d) % phi == 1.

3. Які дані необхідні для виконання операцій Encrypt(), Decrypt(), Sign() та Verify()? Чому кожній із цих операцій не потрібні всі параметри RSA?
Процедури Encrypt(M, e, n) та Verify(M, S, e, n) приймають лише публічні дані - відкритий ключ (e, n). Процедури Decrypt(C, d, n) та Sign(M, d, n) використовують секретний ключ d та модуль n. Інші параметри їм не потрібні з міркувань

безпеки: сторонні учасники мають виконувати шифрування та перевірку підпису без доступу до секретних даних власника, а математично всі чотири операції потребують лише піднесення до степеня за модулем n.

4. Які властивості забезпечуються на різних етапах реалізованого протоколу передачі ключа: конфіденційність та підтвердження справжності відправника? На етапі формування цифрового підпису особистим секретним ключем відправника забезпечується підтвердження справжності автора та цілісність ключа. На етапі шифрування ключа k та підпису S відкритим ключем отримувача забезпечується конфіденційність передачі, оскільки розшифрувати криптограми у відкритому каналі може виключно власник секретного ключа d_B.

5. Які перевірки повинен виконати одержувач у реалізованому протоколі передачі ключа, перш ніж вважати отриманий ключ коректним і справжнім? Після розшифрування ключа k та підпису S власним секретним ключем одержувач повинен переконатися, що ключ входить у допустимий діапазон 0 < k < n_A, а головне - виконати перевірку цифрового підпису за відкритим ключем відправника: S^e_A mod n_A == k. Лише якщо ця рівність виконується, отриманий ключ вважається неспотвореним і дійсно надісланим абонентом A.

Висновок

У ході виконання лабораторної роботи реалізовано криптосистему RSA без використання готових криптографічних бібліотек. Для роботи з великими числами було написано функції швидкого бінарного піднесення до степеня за модулем та розширеного алгоритму Евкліда, а також створено генератор 256-бітних простих чисел на основі пробних ділень і тесту Міллера-Рабіна. Згенеровано ключі для двох абонентів і протестувано всі основні механізми системи: двостороннє шифрування, розшифрування, створення цифрового підпису та протокол захищеної передачі сесійного ключа по відкритому каналу. Коректність роботи нашої програми та її повну сумісність зі стандартом додатково підтверджено успішним обміном зашифрованими даними із зовнішнім тестовим веб-сервером.